



FIAP – Faculdade de Informática e Administração Paulista

Curso de Análise e Desenvolvimento de Sistemas – Turma 1TDSPB

Domain Driven Design Using Java

Sprint 4 - Challenge - ONG Turma do Bem

***BemCare* - Triagem e agendamento em um só fluxo**

Guilherme Barca – RM568517

Juliana de Souza Rodrigues Marques – RM566795

Lucas Gomes Kosio – RM566828

Sumário

Objetivo e escopo do projeto -	3
Descrição da solução -	4
Principais funcionalidades -	5
Explicação dos métodos lógicos -	6
Tabelas de endpoints -	10
Protótipos -	11
MER -	17
Diagrama de Classes -	18
Como rodar a aplicação -	19

Objetivo & escopo do projeto

O Bem Care é um sistema de gestão desenvolvido para apoiar a ONG Turma do Bem, que conecta dentistas voluntários a pacientes de baixa renda em todo o Brasil. O sistema tem como objetivo centralizar o cadastro de dentistas voluntários, o cadastro de pacientes atendidos e o gerenciamento de solicitações de agendamento de consultas odontológicas, substituindo processos manuais por uma solução digital integrada.

O escopo contempla uma API RESTful desenvolvida em Java com o framework Quarkus, integrada a um banco de dados Oracle, e um painel web desenvolvido em React com TypeScript, permitindo o gerenciamento completo das operações da ONG.

Descrição da solução

O Bem Care é uma aplicação web full-stack composta por dois módulos integrados. O backend é uma API RESTful construída com Quarkus (Java), responsável por toda a lógica de negócio e persistência de dados em banco Oracle. O frontend é um painel administrativo construído com React e TypeScript, que consome a API e oferece uma interface visual para os operadores da ONG gerenciarem dentistas, pacientes e agendamentos em tempo real.

Principais funcionalidades

Gestão de Dentistas Voluntários: Permite cadastrar, visualizar, editar e remover dentistas voluntários, armazenando informações como nome, CRO, especialidade, e-mail e cidade de atuação.

Gestão de Pacientes: Permite cadastrar, visualizar, editar e remover pacientes atendidos pela ONG, com dados como nome, idade, responsável, cidade e telefone de contato.

Gestão de Agendamentos: Permite que pacientes ou responsáveis solicitem agendamentos de consultas informando data preferida, período e motivo. Os operadores da ONG podem confirmar, recusar ou remover as solicitações diretamente pelo painel.

Filtro de Agendamentos por Status: Permite buscar agendamentos filtrando por status (Pendente, Confirmado ou Recusado), facilitando o controle operacional da ONG.

Integração Frontend + Backend: O painel web consome a API em tempo real, refletindo imediatamente qualquer alteração feita no banco de dados.

Explicação dos métodos lógicos

Os quatro métodos HTTP exigidos estão implementados nos três recursos da API. Abaixo a explicação de cada um com o trecho de código correspondente.

GET - Listar recursos

Responsável por buscar todos os registros de uma entidade no banco de dados e retorná-los em formato JSON. Utilizado nas três entidades do sistema.

Exemplo em DentistaResource.java:

```
@GET
@Produces(MediaType.APPLICATION_JSON)
public Response selecionarRs() {
    ArrayList<Dentista> lista = dentistaBO.selecionarBo();
    return Response.ok(lista).build();
}
```

Explicação dos métodos lógicos

POST - Inserir novo registro

Responsável por receber um objeto JSON no corpo da requisição e persistir um novo registro no banco de dados. Retorna o status 201 Created com a URI do recurso criado.

Exemplo em PacienteResource.java:

```
@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response inserirRs(Paciente paciente, @Context UriInfo uriInfo) {
    pacienteBO.inserirBo(paciente);
    UriBuilder builder = uriInfo.getAbsolutePathBuilder();
    builder.path(Integer.toString(paciente.getId()));
    return Response.created(builder.build()).build();
}
```

Explicação dos métodos lógicos

PUT - Atualizar registro existente

Responsável por receber um objeto JSON completo e atualizar o registro correspondente no banco de dados.

Exemplo em `AgendamentoResource.java`:

```
@PUT
@Consumes(MediaType.APPLICATION_JSON)
public Response atualizarRs(Agendamento agendamento) {
    agendamentoBO.atualizarBo(agendamento);
    return Response.ok().build();
}
```

Explicação dos métodos lógicos

DELETE - Remover registro

Responsável por remover um registro do banco de dados a partir do ID informado na URL.

Exemplo em DentistaResource.java:

```
@DELETE
@Path("/{id}")
public Response deletarRs(@PathParam("id") int id) {
    dentistaBO.deletarBo(id);
    return Response.ok().build();
}
```

Tabelas de endpoints

Dentista — /dentista

Método	URL	Descrição	Status Sucesso	Status Erro
GET	/dentista	Lista todos os dentistas	200 OK	500 Internal Server Error
GET	/dentista/{id}	Busca dentista por ID	200 OK	404 Not Found / 400 Bad Request
POST	/dentista	Cadastra novo dentista	201 Created	400 Bad Request / 500 Internal Server Error
PUT	/dentista	Atualiza dentista existente	200 OK	400 Bad Request / 500 Internal Server Error
DELETE	/dentista/{id}	Remove dentista por ID	200 OK	400 Bad Request / 500 Internal Server Error

Paciente — /paciente

Método	URL	Descrição	Status Sucesso	Status Erro
GET	/paciente	Lista todos os pacientes	200 OK	500 Internal Server Error
GET	/paciente/{id}	Busca paciente por ID	200 OK	404 Not Found / 400 Bad Request
POST	/paciente	Cadastra novo paciente	201 Created	400 Bad Request / 500 Internal Server Error
PUT	/paciente	Atualiza paciente existente	200 OK	400 Bad Request / 500 Internal Server Error
DELETE	/paciente/{id}	Remove paciente por ID	200 OK	400 Bad Request / 500 Internal Server Error

Agendamento — /agendamento

Método	URL	Descrição	Status Sucesso	Status Erro
GET	/agendamento	Lista todos os agendamentos	200 OK	500 Internal Server Error
GET	/agendamento/{id}	Busca agendamento por ID	200 OK	404 Not Found / 400 Bad Request
GET	/agendamento/status/{status}	Filtra agendamentos por status	200 OK	400 Bad Request / 500 Internal Server Error
POST	/agendamento	Cria novo agendamento	201 Created	400 Bad Request / 500 Internal Server Error
PUT	/agendamento	Atualiza agendamento existente	200 OK	400 Bad Request / 500 Internal Server Error
PATCH	/agendamento/{id}/status	Atualiza apenas o status	200 OK	400 Bad Request / 500 Internal Server Error
DELETE	/agendamento/{id}	Remove agendamento por ID	200 OK	400 Bad Request / 500 Internal Server Error

Protótipo em funcionamento



Tela inicial do sistema.

Protótipo em funcionamento

 Turma do Bem

 Início

 Dentistas

 Pacientes

 Agendamentos

 Sair

 Agendamentos

Solicitar agendamento

Preencha os dados para solicitar um atendimento. Nossa equipe entrará em contato para confirmar.

Nome do paciente

Telefone

(11) 99999-0000

Cidade

Data desejada

dd/mm/aaaa 

Período

Manhã 

Enviar solicitação

Solicitações recebidas

Painel da ONG para confirmar ou recusar pedidos.

LUCAS GOMES KOSIO

(11)91245-2345 · São Paulo

31/07/2026 — Tarde

 Confirmar

 Recusar

 Remover

Pendente

Tela de agendamentos e solicitações onde a ONG pode ter controle de cada agendamento.

Protótipo em funcionamento

Editar Paciente ×

Nome
João Pedro

Idade
12

Responsável
Maria Silva

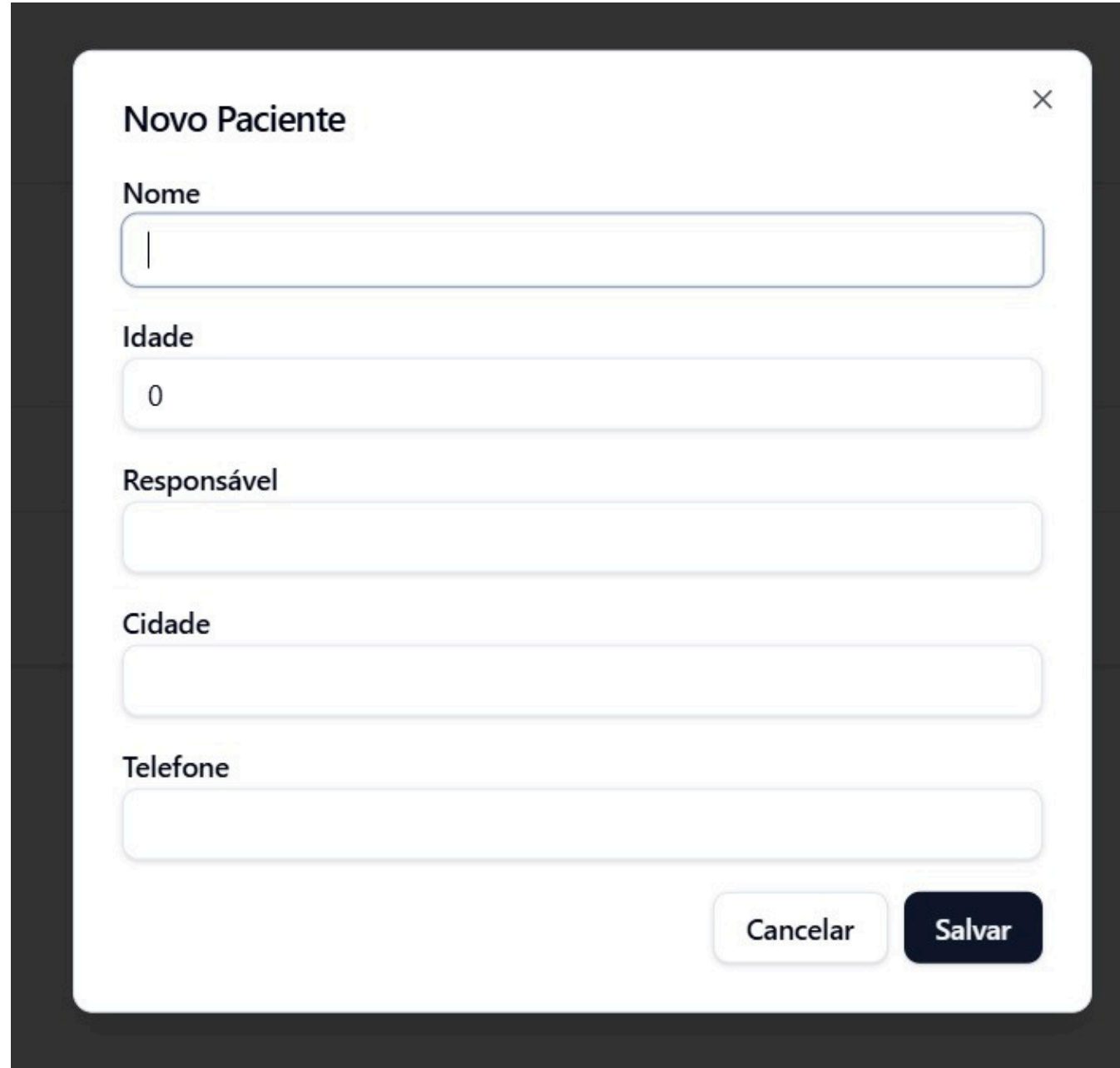
Cidade
São Paulo

Telefone
(11) 99999-0001

Cancelar Salvar

Tela de “editar paciente”.

Protótipo em funcionamento



Novo Paciente

Nome

Idade

Responsável

Cidade

Telefone

Cancelar Salvar

Detailed description: This is a screenshot of a web application prototype for registering a new patient. The interface is contained within a white modal box with rounded corners, set against a dark gray background. At the top left of the modal is the title 'Novo Paciente', and at the top right is a close button represented by a small 'x' icon. Below the title, there are five vertically stacked text input fields. The first field is labeled 'Nome' and contains a single vertical cursor line. The second field is labeled 'Idade' and contains the number '0'. The third field is labeled 'Responsável'. The fourth field is labeled 'Cidade'. The fifth field is labeled 'Telefone'. At the bottom right of the modal, there are two buttons: a light gray button labeled 'Cancelar' and a dark blue button labeled 'Salvar'.

Tela de “cadastrar novo paciente”.

Protótipo em funcionamento

Turma do Bem

Início

Dentistas

Pacientes

Agendamentos

Sair

Dentistas

Voluntários cadastrados na ONG.

+ Novo

Lista

Nome	CRO	Especialidade	E-mail	Cidade	Ações
Dra. Ana Souza	CRO-SP 12345	Ortodontia	ana@turmadobem.org	São Paulo	
Dr. Carlos Lima	CRO-RJ 54321	Odontopediatria	carlos@turmadobem.org	Rio de Janeiro	

Tela de dentistas cadastrados.

Protótipo em funcionamento



A login form for a system called "Turma do Bem". The form is centered on a light gray background. It features a smiley face icon at the top, followed by the title "Turma do Bem" and the subtitle "Acesse o sistema de gestão". Below this are two input fields: "E-mail" with the placeholder text "voce@exemplo.com" and "Senha". A dark blue button labeled "Entrar" is positioned below the password field. At the bottom, a small note states "Demo: qualquer e-mail e senha funcionam."



Turma do Bem

Acesse o sistema de gestão

E-mail

voce@exemplo.com

Senha

Entrar

Demo: qualquer e-mail e senha funcionam.

Tela de login do sistema.

MER

T_BC_USUARIO		
NUMBER	id	PK
VARCHAR2	email	
VARCHAR2	senha	
VARCHAR2	perfil	

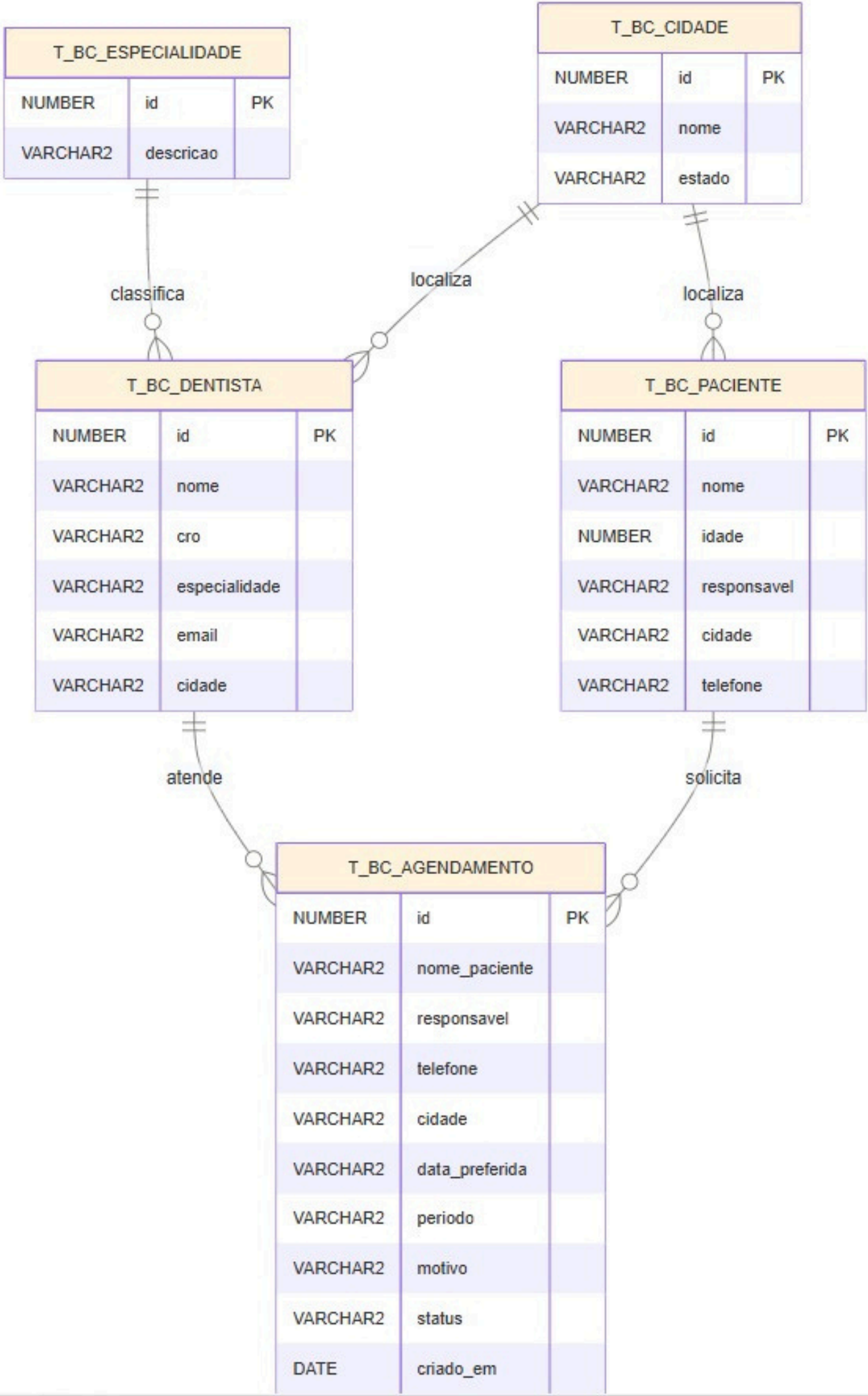
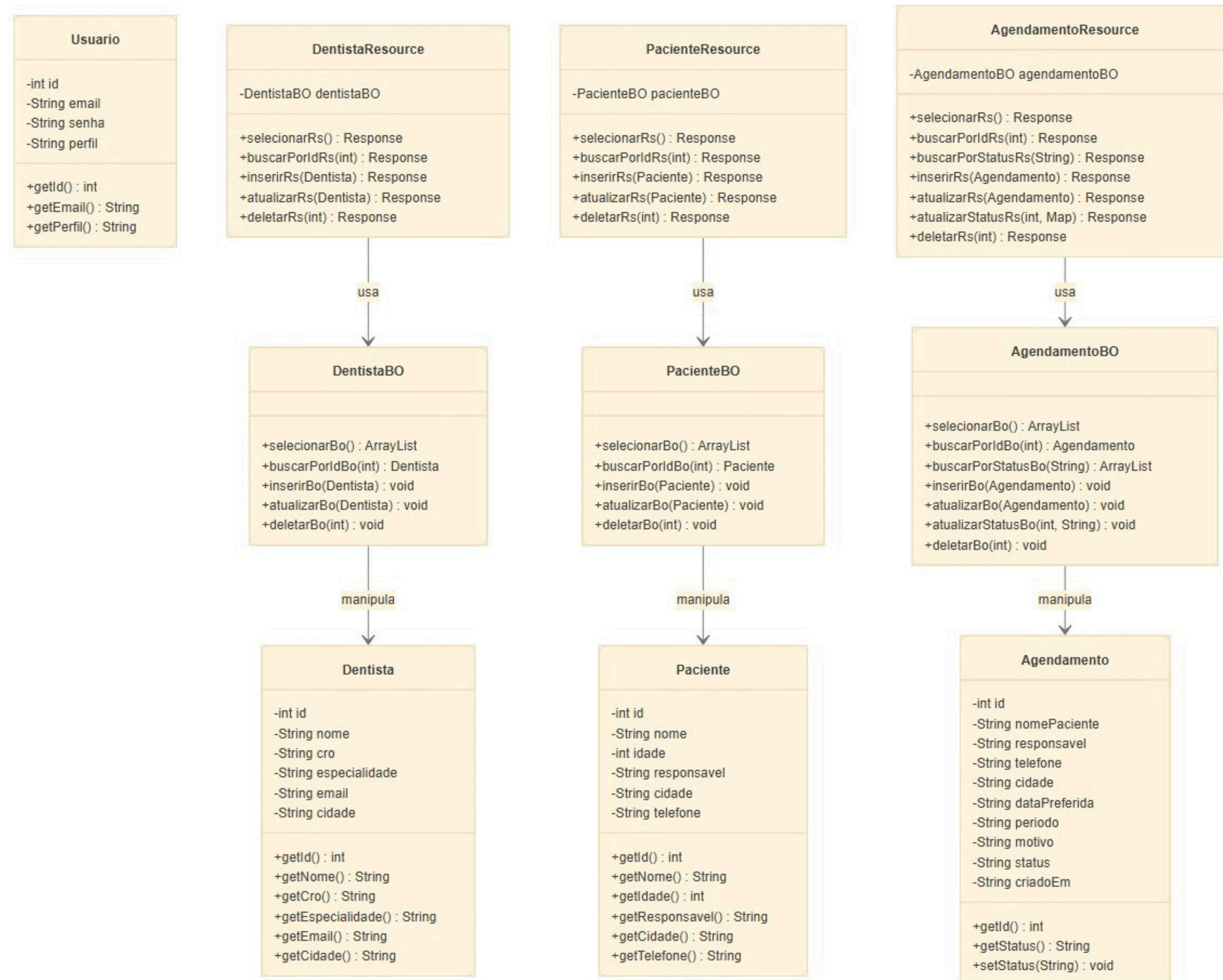


Diagrama de classes



Como rodar a aplicação

Pré-requisitos:

Java 17 ou superior instalado

Maven instalado

Node.js instalado

Banco Oracle acessível com o script SQL executado

IntelliJ IDEA (recomendado para o backend)

VS Code (recomendado para o frontend) e Backend (Quarkus)

1. Abra a pasta backend/ no IntelliJ IDEA
2. Configure as credenciais do banco Oracle no arquivo src/main/resources/application.properties
3. No terminal, dentro da pasta backend/, execute:

```
mvn quarkus:dev
```

A API estará disponível em <http://localhost:8080>

Para verificar, acesse <http://localhost:8080/dentista> no browser — deve retornar um array JSON

Como rodar a aplicação

Frontend (React):

Abra a pasta frontend/ no VS Code

No terminal, dentro da pasta frontend/, execute:

```
npm install  
npm run dev -- --port 5173
```

O painel estará disponível em <http://localhost:5173>

Observação importante:

O backend deve estar rodando antes de iniciar o frontend, pois o painel consome a API em tempo real.